Smart Campus Wi-Fi Monitoring & Network Health Dashboard

Project explanation document

1. What the project is

This web application lets students and staff run a Wi-Fi speed test from their campus location, see a health score, and report problems with the result attached. The IT team monitors network health across buildings, works through complaints, is warned about outages automatically, and gets analytics and intelligent insights.

It is a TypeScript monorepo with three parts that share one language and one set of rules:

Part	Folder	Technology
Shared rules and types	`packages/shared`	TypeScript, Zod
Backend API	`apps/server`	Node.js, Express, SQLite (better-sqlite3), Server-Sent Events
Frontend	`apps/web`	React 18, Vite, Tailwind CSS, TanStack Query, Recharts

`npm run setup && npm start` gives a working system on `http://localhost:3000` with demo data. No cloud services are used.

2. Important folders and files

```
campus-wifi-monitor/
├─ package.json              npm workspaces + all root scripts (setup, start, test, simulate…)
├─ .eslintrc.cjs             lint rules incl. the clean-architecture boundary rule
├─ .env.example              configuration (port, timezone, quick mode, optional local LLM)
├─ docs/                     this document + the architecture blueprint
├─ data/                     SQLite database file (created at runtime)
├─ packages/shared/src/
│  ├─ constants.ts           roles, statuses, complaint categories, status colours, role→permission table
│  ├─ schemas/index.ts       Zod validation schemas used by BOTH server and web forms
│  ├─ types.ts               API data shapes (DTOs)
│  └─ domain/                pure business logic — no database, no clock, no network
│     ├─ health.ts           health score + Excellent/Good/Fair/Poor/Critical
│     ├─ locationHealth.ts   recency-weighted score for a location
│     ├─ outage.ts           outage rules R1–R3 and recovery
│     ├─ complaintFlow.ts    complaint state machine
│     ├─ anomaly.ts          anomaly, problem and trend detection
│     ├─ classifier.ts       complaint text classifier
│     ├─ prediction.ts       outage-risk score + peak-usage forecast
│     ├─ priority.ts         "inspect first" ranking
│     ├─ summary.ts          AI network summary sentences
│     └─ __tests__/          unit tests (incl. the brief's own examples)
├─ apps/server/
│  ├─ migrations/            001_init.sql (schema), 002_…sql
│  ├─ scripts/               migrate, seed, reset, simulate, simulate-outage, simulate-recover
│  ├─ tests/api.test.ts      API integration tests (in-memory database)
│  └─ src/
│     ├─ index.ts            starts the server, scheduler; prints LAN URLs
│     ├─ config.ts           reads .env
│     ├─ container.ts        wires every repository and service together (dependency injection)
│     ├─ http/               Express app, middleware (auth, RBAC, CSRF, rate limits, errors), all API routes
│     ├─ infrastructure/     database, password/JWT, SSE hub, scheduler jobs, optional Ollama adapter
│     ├─ shared/             errors, clock/timezone helpers, ports
│     └─ modules/<name>/     one folder per feature:
```

```
│          application/      use cases / services (business workflow)
│          infrastructure/   SQL repository for that feature
│          http/             (speedtest probe routes)
└─ apps/web/
   ├─ e2e/                   Playwright end-to-end test
   └─ src/
      ├─ api/                fetch wrapper + one query hook per endpoint
      ├─ auth/               session provider, route guards by permission
      ├─ realtime/           live updates (EventSource → refresh data, toasts)
      ├─ components/         layout, buttons, cards, badges, modal, chart frame, location picker
      ├─ lib/format.ts       number/date formatting
      └─ features/           one folder per screen group:
           speedtest/        engine/ (ping, download, upload, runner), gauge, result card
           status/           campus status board and heatmap
           history/          test history
           complaints/       new complaint, my complaints, IT queue, detail + workflow
           outages/          outages and maintenance
           dashboard/        IT dashboard, insights, recommendations, AI summary
           locations/        location detail for IT
           analytics/        analytics charts, compare & reports
           admin/            users & roles, locations, thresholds, activity log, share-on-Wi-Fi QR
           notifications/    bell, notifications page, toasts
```

Module folders on the server (`modules/*`): `auth`, `users`, `locations`, `speedtest`, `tests`, `complaints`, `outages`, `maintenance`, `notifications`, `analytics` (dashboard + analytics), `insights`, `settings`, `activity`. HTTP routes for all modules are grouped by module in `apps/server/src/http/routes.ts`, so the whole API contract can be read in one file.

3. How the speed-test feature works

The test runs in the browser and measures the link between the device and the campus server.

Phase	File	What happens
1. Ping	`apps/web/src/features/speedtest/engine/ping.ts`	20 small requests to `GET /api/speedtest/ping`, 50 ms apart, each with a 1.5 s timeout. Ping = median round-trip time (first warm-up sample dropped); jitter = average change between consecutive samples; packet loss = timed-out probes ÷ probes.
2. Download	`engine/download.ts`	4 parallel streams from `GET /api/speedtest/download?bytes=8MiB` for 8 s. The first 1.5 s (TCP slow-start) is ignored. Mbps = bytes × 8 ÷ seconds ÷ 10^6.
3. Upload	`engine/upload.ts`	3 parallel uploads of random 2 MiB blocks to `POST /api/speedtest/upload` for 6 s, first 1 s ignored.
Orchestration	`engine/runner.ts`	Runs the phases in order, reports live values to the gauge, and maps failures to reasons.
Server probes	`apps/server/src/modules/speedtest/http/speedtest.routes.ts`	Ping returns 204; download streams incompressible random bytes (never compressed or cached); upload counts and discards bytes.

Two targets (`engine/targets.ts`). *Campus network* measures against the probe endpoints above and isolates campus Wi-Fi quality. *Internet* uses Cloudflare's public speed-test endpoints (`speed.cloudflare.com/__down` and `__up`), so results

are comparable with speedtest.net and fast.com. For that target, uploads use plain cross-origin POSTs, so no CORS preflight is needed. The chosen target is stored with each result and shown as an “Internet” badge in history.

When finished, the page sends the raw numbers to `POST /api/tests`. The **server** (not the browser) calculates the score, so results cannot be faked. If a test cannot complete (no ping answers, or download/upload fails), the page shows *“Speed test could not be completed. Please check your connection and try again.”*. Only a failure record is stored, never an incorrect result. If the server is unreachable, the failure is queued and sent later.

4. Where network health calculations are implemented

- **`packages/shared/src/domain/health.ts`**: converts download, upload, ping and packet loss into sub-scores (logarithmic for speeds, linear for ping/loss), combines them with configurable weights (30 / 15 / 30 / 25 %), subtracts a penalty for recent failures and open complaints, and maps the result to Excellent ≥ 90, Good ≥ 70, Fair ≥ 50, Poor ≥ 30, Critical below 30. The brief’s examples are unit tests: 36 / 14 Mbps, 28 ms, 1 % → **Good**; 5.2 / 1.8 Mbps, 190 ms, 10 % → **Poor (≈ 34)**.
- **`packages/shared/src/domain/locationHealth.ts`**: the current status of a location. It uses a recency-weighted average of its last tests (30-minute half-life), marks the status “stale” when there are no recent tests, and “unknown” when there is no data.
- **Called from** `apps/server/src/modules/tests/application/TestService.ts` (on every submitted test) and `modules/locations/application/LocationService.ts` (status refresh, degraded/recovered notifications).
- Administrators change every threshold on **Health thresholds**. That page previews scores with the *same* function the server uses.

5. Where the AI / intelligent features are implemented

All features run locally and deterministically (explainable, no training, no internet). The maths lives in `packages/shared/src/domain/`; `apps/server/src/modules/insights/application/InsightsService.ts` runs it on new tests and every 5 minutes, and stores results in the `insights` table.

Brief feature	Method	File
Automatic problem detection	≥ 3 of the last 5 tests Poor/Critical → “Possible network problem detected in {location}.”	`anomaly.ts` (`detectProblem`)
Performance trend detection	Last 3 scores < 75 % of the 14-day average → “Network performance is lower than the usual average for this location.”	`anomaly.ts` (`detectTrendDrop`)
Network anomaly detection	Robust z-score (median/MAD) against the same hour ±1 over 14 days	`anomaly.ts`
Complaint classification	Weighted keyword/phrase matching with a confidence score; suggests the category live while typing and shows IT an “AI suggests” chip	`classifier.ts`
Outage prediction	Risk 0–100 from score trend (regression slope), failure rate, complaints and anomalies	`prediction.ts` (`outageRisk`)
Peak usage prediction	Hour-of-week profile over 28 days, smoothed, low-score hours flagged as dips for the next 24 h	`prediction.ts` (`forecastNext24h`)
AI network summary	Finds recurring poor hour-ranges and writes sentences such as “Library Floor	`summary.ts`, `infrastructure/llm/llm.ts`

Brief feature	Method	File
	2 experienced high latency and poor download speed between 12 PM and 2 PM for the last three days." Optional local Ollama only rewords the text and never adds facts.	
Problem location detection & improvement recommendation	Priority = current badness + share of poor tests + complaint volume + outages/recurring days → ranked "Inspect first" list with reasons	`priority.ts`
Outage detection (core)	R1: ≥ 3 users, same complaint, same place, 30 min. R2: ≥ 3 unreachable tests from ≥ 2 users in 15 min. R3: last 3 tests Critical. Auto-recovers after 2 good tests.	`outage.ts`, `modules/outages/application/OutageService.ts`

6. How frontend, backend, database and other components connect

```
Browser (React app)
   │  fetch /api/...  (JSON, session cookie, CSRF header)
   │  EventSource /api/events  (live updates)
   ▼
Express server  ── http/middleware: auth → permission check → validation (Zod)
   │
   ├─ modules/*/application   services: SubmitSpeedTest pipeline, complaints workflow, outages, insights…
   │        uses pure rules from packages/shared/domain
   ├─ modules/*/infrastructure SQL repositories (prepared statements)
   │           ▼
   │      SQLite database file  data/campus-wifi.db
   ├─ SSE hub  → pushes dashboard.updated / notification / outage.opened… to connected browsers
   └─ Scheduler (every 60 s / 5 min) → refresh statuses, evaluate outages, insights, maintenance notices
```

Request example (speed test): the browser measures → `POST /api/tests` → validation → `TestService.submit` calculates the health score → saves the row → refreshes the location status → checks outage rules → runs insights → commits the transaction → the SSE hub tells every open dashboard to refresh. That is why the IT dashboard and heatmap update without reloading.

Security: passwords are hashed with bcrypt. The session is an httpOnly cookie (JWT, 8 h). Every route requires sign-in except login/register/health, and each route checks a permission from the fixed role table (`constants.ts`). All inputs are validated with Zod, all SQL uses bound parameters, and there are rate limits on login, tests and complaints. Every staff/admin action is written to the activity log.

Shared code: the same Zod schemas validate forms in the browser and requests on the server, and the same health function scores tests on the server and previews thresholds in the admin page.

7. Users and roles

Role	Can do
Student / Staff	Select location, run speed test, view result and history, submit complaints, view outages and campus status
IT Support Staff	All of the above, plus view all tests and complaints, review/assign, investigate/resolve (when assignee), add notes, schedule maintenance, resolve outages, dashboard, insights, analytics, compare buildings, recurring problems
Network / IT Manager	All of the above, plus manage locations, IT staff activity, reports/CSV, activity log, read thresholds

Role	Can do
Administrator	Everything, plus manage users and roles, suspend accounts, configure health thresholds and outage rules, share-on-Wi-Fi QR code

8. Testing

- `npm test`: 33 automated tests covering the domain rules (including the brief's examples), the API (authentication, permissions, the complaint workflow, outage open/recover, maintenance suppression, analytics, insights) and the speed-test engine.
- `npm run test:e2e`: a browser test in which a student runs a test and files a complaint, IT moves it to Resolved, the dashboard count changes, the student is notified, and the manager opens analytics.